

# 《语法分析程序的设计与实现》实验报告

## 1 需求分析

### 1.1 任务描述

采用LL(1)和LR(1)两种分析方法，编写语法分析程序，对算术表达式进行语法分析。算术表达式文法G定义如下：

1 |  $G = (V_N, V_T, P, E)$

其中：

$$G = (V_N, V_T, P, E) \quad (1)$$

其中

$$V_N = \{E, T, F\}$$

$$V_T = \{n, +, -, *, /, (, )\}$$

$$P = \{E \rightarrow E + T, E \rightarrow E - T, E \rightarrow T, T \rightarrow T * F, T \rightarrow T / F, T \rightarrow F, F \rightarrow (E), F \rightarrow n, \}$$

### 1.2 输入格式

从标准输入读入数据，输入为一个算术表达式，构成该算术表达式的字符有： $\{n, +, -, *, /, (, )\}$

### 1.3 输出格式

要求输出到标准输出中。

对于LL(1)分析：

- 每行包含制表符(\t)分隔的三个部分：分析栈：以\$符号表示栈底的字符串 输入串栈：以\$符号表示栈底的字符串 分析动作：产生式编号或 match 或 error 或 accept

对于LR(1)分析：

- 每行仅包含一个部分：归约使用的产生式编号或 shift 或 error 或 accept

## 2 程序设计与实现

### 2.1 数据结构

#### 2.1.1 文法相关数据结构

终结符和非终结符集合：

```

1 set<char> N = {'A', 'B', 'T', 'F', 'E'}; // 非终结符集合
2 set<char> T = {'+', '-', '*', '/', '(', ')', '\n'}; // 终结符集合
3 char S = 'E'; // 开始符号

```

产生式定义:

```

1 // LL(1)文法产生式 (消除左递归后)
2 pair<char, string> P[10] = {
3     {'E', "TA"},
4     {'A', "+TA"},
5     {'A', "-TA"},
6     {'A', ""},
7     {'T', "FB"},
8     {'B', "*FB"},
9     {'B', "/FB"},
10    {'B', ""},
11    {'F', "(E)"},
12    {'F', "n"},
13 };
14
15 // LR(1)文法产生式 (增广文法)
16 pair<char, string> P[9] = {
17     {'S', "E"},
18     {'E', "E+T"},
19     {'E', "E-T"},
20     {'E', "T"},
21     {'T', "T*F"},
22     {'T', "T/F"},
23     {'T', "F"},
24     {'F', "(E)"},
25     {'F', "n"},
26 };

```

## 2.1.2 分析表数据结构

LL(1)预测分析表:

```

1 unordered_map<char, set<char>> first; // FIRST集
2 unordered_map<char, set<char>> follow; // FOLLOW集
3 map<pair<char, char>, int> table; // 预测分析表
4 #define SYNCH -1 // 同步标记

```

LR(1)分析表:

```

1 map<pair<int, char>, int> GOTO; // GOTO表
2 map<pair<int, char>, pair<string, int>> ACTION; // ACTION表

```

## 2.1.3 LR(1)项目相关数据结构

LR(1)项目定义:

```

1 struct project {
2     char l; // 产生式左部
3     string r; // 产生式右部
4     set<char> charcon; // 向前看符号集
5     int dosPos = -1; // 圆点位置
6     int type; // 项目类型
7
8     // 重载相等运算符
9     bool operator==(const project& other) const {
10         return l == other.l && r == other.r &&
11             charcon == other.charcon && dosPos == other.dosPos;
12     }
13 };

```

项目集规范族：

```

1 vector<vector<project>> I(100); // 项目集族
2 int countI = 1; // 项目集计数器

```

## 2.1.4 分析栈数据结构

LL(1)分析栈：

```

1 char st[100]; // 分析栈
2 int top; // 栈顶指针

```

LR(1)分析栈：

```

1 int st[100]; // 状态栈
2 int top; // 栈顶指针

```

## 2.1.5 输入处理数据结构

字符串流处理：

```

1 istream iss(s); // 输入字符串流
2 char c; // 当前输入字符

```

## 2.2 LL(1)语法分析器

### 2.2.1 文法设计

理论定义：

LL(1)分析法属于自顶向下的语法分析方法，要求文法满足LL(1)条件。对于原文法：

$$\begin{aligned}
 E &\rightarrow E + T \mid E - T \mid T \\
 T &\rightarrow T * F \mid T / F \mid F \\
 F &\rightarrow (E) \mid n
 \end{aligned}$$

该文法存在**左递归**和**左公因子**问题，不满足LL(1)条件。根据编译原理理论，需要进行文法变换：

1. **消除直接左递归**：对于形如 $A \rightarrow A\alpha \mid \beta$ 的产生式，变换为： $A \rightarrow \beta A'$   $A' \rightarrow \alpha A' \mid \epsilon$
2. **提取左公因子**：对于有共同前缀的产生式进行因子提取。

**形式化描述：**

设原文法 $G=(V_N, V_T, P, S)$ ，经过消除左递归和提取左公因子后得到新文法 $G'=(V_N', V_T, P', S)$ ，其中：

- $V_N' = V_N \cup \{A, B\}$ （新增非终结符）
- $P'$ 为消除左递归后的产生式集合

对原文法进行消除左递归和提取左公因子处理，得到LL(1)文法：

```

1  pair<char, string> P[10] = {
2      {'E', "TA"},
3      {'A', "+TA"},
4      {'A', "-TA"},
5      {'A', ""},
6      {'T', "FB"},
7      {'B', "*FB"},
8      {'B', "/FB"},
9      {'B', ""},
10     {'F', "(E)"},
11     {'F', "n"},
12 };

```

## 2.2.2 FIRST集和FOLLOW集计算

实现了完整的FIRST集和FOLLOW集计算算法：

## 理论定义:

**FIRST集定义:** 对于任意文法符号串 $\alpha$ ,  $FIRST(\alpha)$ 是可以从 $\alpha$ 推导出的所有串的第一个终结符的集合, 且

$$FIRST(\alpha) = \{a \in V_T \mid \alpha \xRightarrow{*} a\beta\} \cup \{\epsilon \mid \alpha \xRightarrow{*} \epsilon\}$$

**FOLLOW集定义:** 对于非终结符 $A$ ,  $FOLLOW(A)$ 是所有句型中紧跟在 $A$ 后面的终结符的集合, 即:

$$FOLLOW(A) = \{a \in V_T \mid S \xRightarrow{*} \alpha A a \beta\} \cup \{\$ \mid S \xRightarrow{*} \alpha A\}$$

## 算法原理:

FIRST集的计算基于以下规则:

1. 如果 $X$ 是终结符, 则 $FIRST(X) = \{X\}$
2. 如果 $X$ 是非终结符且有产生式 $X \rightarrow \epsilon$ , 则 $\epsilon \in FIRST(X)$
3. 如果 $X$ 是非终结符且有产生式 $X \rightarrow Y_1 Y_2 \cdots Y_k$ :
  - 将 $FIRST(Y_1)$ 中除 $\epsilon$ 外的所有符号加入 $FIRST(X)$
  - 如果 $\epsilon \in FIRST(Y_1)$ , 则将 $FIRST(Y_2)$ 中除 $\epsilon$ 外的所有符号加入 $FIRST(X)$
  - 依此类推, 如果所有 $Y_i$ 都能推出 $\epsilon$ , 则将 $\epsilon$ 加入 $FIRST(X)$

## FIRST

```
1  set<char> FIRST(char c) {
2      if (c == '\0' || c == '$' || T.find(c) != T.end()) return {c};
3      if (!first.empty()) {
4          return first[c];
5      }
6      for (auto n : N) {
7          first[n] = {};
8      }
9      unordered_map<char, set<char>> old;
10     while (old != first) {
11         old = first;
12         for (auto p : P) {
13             set<char> tem = FIRST(p.second);
14             first[p.first].insert(tem.begin(), tem.end());
15         }
16     }
17     return first[c];
18 }

19
20 set<char> FIRST(string s) {
21     set<char> t1 = {'\0'};
22     for (auto x : s) {
23         if (t1.find('\0') == t1.end()) break;
```

```

24     t1.erase('\0');
25     set<char> t2 = FIRST(x);
26     t1.insert(t2.begin(), t2.end());
27 }
28 return t1;
29 }

```

## FOLLOW

### FOLLOW集计算算法:

1. 将\$加入**FOLLOW(S)**, 其中S是开始符号
2. 如果有产生式 $A \rightarrow a B \beta$ , 则将**FIRST( $\beta$ )**中除 $\epsilon$ 外的所有符号加入**FOLLOW(B)**
3. 如果有产生式 $A \rightarrow a B$ 或 $A \rightarrow a B \beta$ 且 $\beta \Rightarrow^* \epsilon$ , 则将**FOLLOW(A)**中的所有符号加入**FOLLOW(B)**

```

1  set<char> FOLLOW(char c) {
2      if (!follow.empty()) {
3          return follow[c];
4      }
5      for (auto x : N) {
6          follow[x] = {};
7      }
8      follow[S].insert('$');
9      unordered_map<char, set<char>> old;
10     while (old != follow) {
11         old = follow;
12         for (auto p : P) {
13             for (int i = 0; i < p.second.size(); i++) {
14                 auto A = p.second[i];
15                 auto tmp = FIRST(&p.second[i + 1]);
16                 follow[A].insert(tmp.begin(), tmp.end());
17                 if (follow[A].count('\0')) {
18                     follow[A].erase('\0');
19                     tmp = FOLLOW(p.first);
20                     follow[A].insert(tmp.begin(), tmp.end());
21                 }
22             }
23         }
24     }
25     return follow[c];
26 }

```

## 2.2.3 预测分析表构建

1. 对每个产生式 $A \rightarrow a$ , 计算**FIRST( $a$ )**
2. 对于每个 $a \in \text{FIRST}(a)$ 且 $a \neq \epsilon$ , 将 $A \rightarrow a$ 加入 $M[A, a]$
3. 如果 $\epsilon \in \text{FIRST}(a)$ , 则对每个 $b \in \text{FOLLOW}(A)$ , 将 $A \rightarrow a$ 加入 $M[A, b]$

构建LL(1)预测分析表:

```

1  void creatTable() {
2      for (int i = 0; i < 10; i++) {
3          pair<char, string> p = P[i];

```

```

4      auto t1 = FIRST(p.second);
5      auto t2 = FOLLOW(p.first);
6      for (auto x : t1) {
7          if (x != '\0') table[{p.first, x}] = i + 1;
8          else for (auto a : t2) table[{p.first, a}] = i + 1;
9      }
10     }
11     // 添加同步信息
12     for (auto x : N) for (auto y : follow[x])
13         if (table[{x, y}] == 0) table[{x, y}] = SYNCH;
14 }

```

## 2.2.4 分析过程实现

### 理论定义：

LL(1)分析使用下推自动机模型，包含：

- **分析栈**：存储待处理的文法符号
- **输入缓冲区**：存储待分析的输入串
- **控制程序**：根据分析表和当前状态决定分析动作

### 分析算法：

1. 初始时，栈底为\$，栈顶为开始符号S
2. 根据栈顶符号X和当前输入符号a选择动作：如果 $X=a=\$$ ，分析成功如果 $X=a\neq \$$ ，匹配成功，弹出X并读入下一个符号如果 $X\in V T$ 但 $X\neq a$ ，报告错误如果 $X\in V N$ ，查表 $M[X,a]$ ：如果 $M[X,a]$ 为空，报告错误否则使用相应产生式进行推导

实现了完整的LL(1)分析过程：

```

1  void LL1(string s) {
2      s += '$';
3      istream iss(s);
4      char c; iss >> c;
5      char st[100]; st[0] = '$'; st[1] = S;
6      int top = 1;
7
8      while (1) {
9          if (st[top] == '$' && c == '$') {
10             cout << "accept" << endl; break;
11          }
12          else if (st[top] == c) {
13             // 匹配处理
14             top--; iss >> c;
15          }
16          else if (table[{st[top], c}] != 0 && table[{st[top], c}] != SYNCH) {
17             // 产生式应用
18             string r = P[table[{st[top], c}] - 1].second;
19             top--;
20             for (int i = r.length() - 1; i >= 0; i--)
21                 st[++top] = r[i];
22          }
23          else {

```

```

24         // 错误处理
25         cout << "error" << endl;
26         iss >> c;
27     }
28 }
29 }

```

## 2.2.5 错误恢复

LL(1)分析中的错误恢复策略包括：

1. **恐慌模式**：发现错误时，跳过输入符号直到找到同步符号
2. **同步符号集**： **$FOLLOW(A)$** 中的符号可以作为非终结符A的同步符号

同步标记策略：

- 对于 $A \in V_N$ 和 $a \in FOLLOW(A)$ ，如果 $M[A, a]$ 为空，则设置 $M[A, a] = SYNCH$
- 遇到 **$SYNCH$** 时，弹出栈顶非终结符A，继续分析

```

1 void creatTable() {
2     ...
3     // 添加同步信息
4     for (auto x : N) for (auto y : follow[x])
5         if (table[{x, y}] == 0) table[{x, y}] = SYNCH;
6 }

```

```

1 void LL1(string s) {
2     else if (table[{st[top], c}] == SYNCH) {
3         printstack(st, top);
4         string rest = iss.str().substr(iss.tellg());
5         cout << '\t' << c << rest << '\t' << "synch" << endl;
6         top--;
7     }
8 }

```

## 2.2.6 LL(1)分析表

**FIRST**和**FOLLOW**集

|          | <i>FIRST</i>     | <i>FOLLOW</i>     |
|----------|------------------|-------------------|
| <i>E</i> | (, <i>n</i>      | \$, )             |
| <i>A</i> | +, -, $\epsilon$ | \$, )             |
| <i>T</i> | (, <i>n</i>      | \$, +, -, )       |
| <i>B</i> | *, /, $\epsilon$ | \$, +, -, )       |
| <i>F</i> | (, <i>n</i>      | \$, +, -, *, /, ) |

分析表



|   | ( | )  | *  | +  | -  | /  | n  | \$ |
|---|---|----|----|----|----|----|----|----|
| A | 0 | 4  | 0  | 2  | 3  | 0  | 0  | 4  |
| B | 0 | 8  | 6  | 8  | 8  | 7  | 0  | 8  |
| E | 1 | -1 | 0  | 0  | 0  | 0  | 1  | -1 |
| F | 9 | -1 | -1 | -1 | -1 | -1 | 10 | -1 |
| T | 5 | -1 | 0  | -1 | -1 | 0  | 5  | -1 |

产生式编号说明

| 编号 | 产生式                      | 说明         |
|----|--------------------------|------------|
| 0  | 空项（错误）                   | 语法错误       |
| 1  | $E \rightarrow TA$       | 表达式推导      |
| 2  | $A \rightarrow +TA$      | 加法运算       |
| 3  | $A \rightarrow -TA$      | 减法运算       |
| 4  | $A \rightarrow \epsilon$ | A的空产生式     |
| 5  | $T \rightarrow FB$       | 项推导        |
| 6  | $B \rightarrow *FB$      | 乘法运算       |
| 7  | $B \rightarrow /FB$      | 除法运算       |
| 8  | $B \rightarrow \epsilon$ | B的空产生式     |
| 9  | $F \rightarrow (E)$      | 括号表达式      |
| 10 | $F \rightarrow n$        | 数值         |
| -1 | <b>synch</b>             | 同步标记（错误恢复） |

文法产生式完整列表

|    |                             |
|----|-----------------------------|
| 1  | 1. $E \rightarrow TA$       |
| 2  | 2. $A \rightarrow +TA$      |
| 3  | 3. $A \rightarrow -TA$      |
| 4  | 4. $A \rightarrow \epsilon$ |
| 5  | 5. $T \rightarrow FB$       |
| 6  | 6. $B \rightarrow *FB$      |
| 7  | 7. $B \rightarrow /FB$      |
| 8  | 8. $B \rightarrow \epsilon$ |
| 9  | 9. $F \rightarrow (E)$      |
| 10 | 10. $F \rightarrow n$       |

分析表解读

## 合法分析动作

- **E**: 遇到 `(` 或 `n` 时使用产生式1 ( $E \rightarrow TA$ )
- **A**: 遇到 `+` 时使用产生式2 ( $A \rightarrow +TA$ ) 遇到 `-` 时使用产生式3 ( $A \rightarrow -TA$ ) 遇到 `)` 或 `$` 时使用产生式4 ( $A \rightarrow \epsilon$ )
- **T**: 遇到 `(` 或 `n` 时使用产生式5 ( $T \rightarrow FB$ )
- **B**: 遇到 `*` 时使用产生式6 ( $B \rightarrow *FB$ ) 遇到 `/` 时使用产生式7 ( $B \rightarrow /FB$ ) 遇到 `+`、`-`、`)`、`$` 时使用产生式8 ( $B \rightarrow \epsilon$ )
- **F**: 遇到 `(` 时使用产生式9 ( $F \rightarrow (E)$ ) 遇到 `n` 时使用产生式10 ( $F \rightarrow n$ )

## 错误处理

- **0**: 表示语法错误, 需要错误恢复
- **-1**: 同步标记 (synch), 用于错误恢复时弹出栈顶非终结符

## 2.3 LR(1)语法分析器

### 2.3.1 文法设计

LR(1)分析法属于自底向上的语法分析方法, 能够处理更大一类上下文无关文法。为了构造LR分析器, 需要对原文法进行**增广** (Augmentation)。

**增广文法定义**: 对于文法  $G=(V_N, V_T, P, S)$ , 其增广文法  $G'=(V_N \cup \{S'\}, V_T, P \cup \{S' \rightarrow S, S'\})$ , 其中  $S' \notin V_N$  是新的开始符号。

**增广的目的**:

1. 使文法有一个唯一的开始符号产生式
2. 确保分析器在归约到  $S'$  时接受输入
3. 简化分析表的构造

**形式化描述**:

设原文法  $G=(V_N, V_T, P, E)$ , 其中:

- $V_N=\{E, T, F\}$
- $V_T=\{n, +, -, *, /, (, )\}$
- $P$  为原文法产生式集合

增广后得到新文法  $G'=(V_N \cup \{S\}, V_T, P', S)$ , 其中  $P'$  包含:

```
1 pair<char, string> P[9] = {
2     {'S', "E"},
3     {'E', "E+T"},
4     {'E', "E-T"},
5     {'E', "T"},
6     {'T', "T*F"},
7     {'T', "T/F"},
8     {'T', "F"},
9     {'F', "(E)"},
10    {'F', "n"},
11 };
```

### 2.3.2 LR(1)项目定义

**LR(1)项目** (Item) 是一个三元组  $[A \rightarrow \alpha \cdot \beta, a]$ , 其中:

- $A \rightarrow a\beta$  是一个产生式
- $\cdot$  表示分析进度（圆点位置）
- $a \in V_T \cup \{\$ \}$  是向前看符号（Lookahead）

项目分类：

1. **移进项目**： $[A \rightarrow a \cdot a \beta, b]$ ，其中  $a \in V_T$
2. **待约项目**： $[A \rightarrow a \cdot B \beta, b]$ ，其中  $B \in V_N$
3. **归约项目**： $[A \rightarrow a \cdot, a]$ ，表示可以按产生式  $A \rightarrow a$  归约
4. **接受项目**： $[S' \rightarrow S \cdot, \$]$

定义了LR(1)项目结构：

```
1 struct project {
2     char l;
3     string r;
4     set<char> charcon;
5     int dosPos = -1;
6     int type;
7     bool operator==(const project& other) const;
8 };
```

### 2.3.3 项目集规范族构建

实现了closure和Goto函数：

**闭包运算  $CLOSURE(I)$** ：对于项目集  $I$ ，其闭包是通过以下规则能够从  $I$  中项目推导出的所有项目的集合：

**闭包计算规则：**

1.  $I \subseteq CLOSURE(I)$
2. 如果  $[A \rightarrow a \cdot B \beta, a] \in CLOSURE(I)$  且  $B \rightarrow \eta$  是产生式，则对于每个  $b \in FIRST(\beta a)$ ，将  $[B \rightarrow \cdot \eta, b]$  加入  $CLOSURE(I)$

**算法原理**：闭包运算实质上是待约项目展开，预测所有可能的推导路径。

**closure**

```
1 void closure(int ind) {
2     queue<project> proQ;
3     for (int i = 0; i < I[ind].size(); i++) {
4         proQ.push(I[ind][i]);
5     }
6     while (!proQ.empty()) {
7         project curpro = proQ.front();
8         if (curpro.dosPos == curpro.r.length() || T.count(curpro.r[curpro.dosPos])) {
9             proQ.pop();
10            continue;
11        }
12        char c = curpro.r[curpro.dosPos];
13        for (auto& p : P) {
14            if (p.first != c) continue;
15
16            project newPro;
17            newPro.dosPos = 0;
```

```

18         newPro.l = p.first;
19         newPro.r = p.second;
20         if (T.count(newPro.r[newPro.dosPos])) newPro.type = 2;
21         if(N.count(newPro.r[newPro.dosPos])) newPro.type = 3;
22         if (newPro.dosPos == newPro.r.length()) newPro.type = 4;
23
24         string beta;
25         if (curpro.dosPos == curpro.r.length() - 1) {
26             newPro.charcon.insert(curpro.charcon.begin(), curpro.charcon.end());
27         }
28         else if (T.count(curpro.r[curpro.dosPos + 1])) {
29             newPro.charcon.insert(curpro.r[curpro.dosPos + 1]);
30         }
31         else {
32             beta = curpro.r.substr(curpro.dosPos + 1, curpro.r.length() -
curpro.dosPos - 1);
33             set<char> tem = FIRST(beta);
34             newPro.charcon.insert(tem.begin(), tem.end());
35         }
36
37         int flag = -1;
38         for (int i = 0; i < I[ind].size(); i++) {
39             if (newPro.l == I[ind][i].l && newPro.r == I[ind][i].r &&
newPro.dosPos == I[ind][i].dosPos) {
40                 flag = i;
41             }
42         }
43
44         if (flag == -1) {
45             proQ.push(newPro);
46             I[ind].push_back(newPro);
47         }
48         else {
49             size_t old_size = I[ind][flag].charcon.size();
50             I[ind][flag].charcon.insert(newPro.charcon.begin(),
newPro.charcon.end());
51
52             if (I[ind][flag].charcon.size() > old_size) {
53                 proQ.push(I[ind][flag]);
54             }
55         }
56     }
57     }
58     proQ.pop();
59 }
60 }

```

## Goto

**Goto函数**  $GOTO(I, X)$ : 对于项目集 $I$ 和文法符号 $X \in V_N \cup V_T$ , 定义:

1 |  $GOTO(I, X) = \text{CLOSURE}(\{[A \rightarrow \alpha X \beta, a] \mid [A \rightarrow \alpha \cdot X \beta, a] \in I\})$

**算法原理：** GOTO函数表示在状态*i*下遇到符号*X*后转移到的新状态。

```
1  int Goto(int ind, char c) {
2      int len = I[ind].size();
3      for (int i = 0; i < len; i++) {
4          string r = I[ind][i].r;
5          int dot = I[ind][i].dosPos;
6          if (dot < r.length() && c == r[dot]) {
7              project tem = I[ind][i];
8              tem.dosPos++;
9              if (tem.dosPos == tem.r.length()) {
10                 tem.type = 4;
11                 I[countI].push_back(tem);
12             }
13             else if (T.count(tem.r[tem.dosPos])) {
14                 tem.type = 2;
15                 I[countI].push_back(tem);
16             }
17             else if (N.count(tem.r[tem.dosPos])) {
18                 tem.type = 3;
19                 I[countI].push_back(tem);
20                 closure(countI);
21             }
22         }
23     }
24
25     countI++;
26     int flag = isMerge();
27     if (flag != -1) {
28         I[countI - 1].clear();
29         countI--;
30         return flag;
31     }
32     else return countI - 1;
33 }
34
```

### 2.3.4 分析表构建

LR(1)分析表包含两个部分：

1. **ACTION表**： **ACTION**[*i*,*a*]，表示在状态*i*下面临输入符号*a*时应采取的动作
2. **GOTO表**： **GOTO**[*i*,*A*]，表示在状态*i*下归约出非终结符*A*后应转移到的状态

**ACTION表动作类型：**

- **移进**  $S_j$ ：将状态*j*压栈，读入下一符号
- **归约**  $r_k$ ：按产生式*k*归约，弹出右部符号，按GOTO表转移
- **接受** *acc*：分析成功
- **报错**：空白表项

**算法原理：**

基于LR(1)项目集规范族构造分析表：

构建ACTION表和GOTO表:

```
1 void PPSS() {
2     initI0();
3     closure(0);
4     vector<project> C = I[0];
5     int curStatus = 0;
6     queue<char> charToRead;
7     map<char, int> sm;
8
9     for (int i = 0; i < I[0].size(); i++) {
10         char c = I[0][i].r[I[0][i].dosPos];
11         if (sm[c] == 0) {
12             charToRead.push(c);
13             sm[c]++;
14         }
15     }
16     charToRead.push('#');
17
18     while (!charToRead.empty()) {
19         if (charToRead.front() == '#') {
20             for (int i = 0; i < I[curStatus].size(); i++) {
21                 if (I[curStatus][i].dosPos == I[curStatus][i].r.length()) {
22                     project tem = I[curStatus][i];
23                     int pos = -1;
24                     for (int j = 0; j < 9; j++) {
25                         pair<char, string> p = P[j];
26                         if (p.first == tem.l && p.second == tem.r) pos = j;
27                     }
28
29                     if (pos == 0) ACTION[{curStatus, '$'}] = { "ACC",-1 };
30                     else {
31                         for (auto x : tem.charcon) {
32                             ACTION[{curStatus, x}] = { "R",pos };
33                         }
34                     }
35                 }
36                 continue;
37             }
38
39             curStatus++;
40             charToRead.pop();
41             continue;
42         }
43
44         int next = Goto(curStatus, charToRead.front());
45         if (T.count(charToRead.front())) {
46             ACTION[{curStatus, charToRead.front()}] = { "S",next };
47         }
48         else if (N.count(charToRead.front())) {
49             GOTO[{curStatus, charToRead.front()}] = next;
50         }
51     }
```

```

52     if (next == countI - 1) {
53         sm.clear();
54         for (int i = 0; i < I[next].size(); i++) {
55             if (I[next][i].dosPos == I[next][i].r.length()) continue;
56             char c = I[next][i].r[I[next][i].dosPos];
57             if (sm[c] == 0) {
58                 charToRead.push(c);
59                 sm[c]++;
60             }
61         }
62         charToRead.push('#');
63     }
64
65     charToRead.pop();
66 }
67
68 }

```

### 2.3.5 分析过程实现

LR(1)分析算法使用下推自动机模型：

**分析栈：** 存储状态序列  $s_0s_1\cdots s_m$

**输入缓冲区：** 剩余输入符号  $a_i a_{i+1}\cdots a_n\$$

**分析表：** 驱动分析过程

**算法步骤：**

1. 初始时，栈中含 $s_0$ ，输入为 $w$
2. 根据栈顶状态 $s_m$ 和输入符号 $a_i$ 查ACTION表：
  - 如果 $ACTION[s_m, a_i] = S_j$ ，移进：压入 $s_j$ ，读入下一符号
  - 如果 $ACTION[s_m, a_i] = r_k$ （归约 $A \rightarrow \beta$ ）：
    - 弹出 $|\beta|$ 个状态
    - 设新栈顶为 $s_t$ ，查 $GOTO[s_t, A] = s_u$
    - 压入 $s_u$
  - 如果 $ACTION[s_m, a_i] = acc$ ，分析成功
  - 如果 $ACTION[s_m, a_i]$ 为空，报错

实现了LR(1)分析过程：

```

1 void LR1(string in) {
2     in += '$';
3     istringstream iss(in);
4     char c; int st[100]; st[0] = 0; int top = 0;
5     iss >> c;
6
7     while (1) {
8         if (ACTION[{ st[top], c }] != make_pair<string,int>("",0)) {
9             if (ACTION[{ st[top], c }].first == "S") {

```

```

10         // 移进动作
11         st[++top] = ACTION[{st[top], c}].second;
12         iss >> c;
13     }
14     else if (ACTION[{st[top], c}].first == "R") {
15         // 归约动作
16         auto p = P[ACTION[{st[top], c}].second];
17         top -= p.second.length();
18         st[++top] = GOTO[{st[top], p.first}];
19     }
20     else if (ACTION[{st[top], c}].first == "ACC") {
21         cout << "accept" << endl; break;
22     }
23 }
24 else { cout << "error" << endl; break; }
25 }
26 }

```

### 2.3.6 LR(1)分析表

#### ACTION表



| 状态 | \$    | +   | -   | *   | /   | (   | )   | n   |
|----|-------|-----|-----|-----|-----|-----|-----|-----|
| 0  | ACC-1 | 0   | 0   | 0   | 0   | S6  | S7  | 0   |
| 1  | 0     | 0   | 0   | 0   | 0   | 0   | 0   | 0   |
| 2  | R3    | 0   | 0   | S8  | 0   | R3  | R3  | S9  |
| 3  | R6    | 0   | 0   | R6  | R6  | R6  | R6  | R6  |
| 4  | 0     | S13 | 0   | 0   | 0   | 0   | 0   | S14 |
| 5  | R8    | 0   | 0   | R8  | R8  | R8  | R8  | R8  |
| 6  | 0     | S4  | 0   | 0   | 0   | 0   | 0   | S5  |
| 7  | 0     | S4  | 0   | 0   | 0   | 0   | 0   | S5  |
| 8  | 0     | S4  | 0   | 0   | 0   | 0   | 0   | S5  |
| 9  | 0     | S4  | 0   | 0   | 0   | 0   | 0   | S5  |
| 10 | 0     | 0   | S19 | 0   | S20 | S21 | 0   | 0   |
| 11 | 0     | 0   | R3  | S22 | R3  | R3  | S23 | 0   |
| 12 | 0     | 0   | R6  | R6  | R6  | R6  | R6  | R6  |
| 13 | 0     | S13 | 0   | 0   | 0   | 0   | 0   | S14 |
| 14 | 0     | 0   | R8  | R8  | R8  | R8  | R8  | R8  |
| 15 | R1    | 0   | 0   | S8  | 0   | R1  | R1  | S9  |
| 16 | R2    | 0   | 0   | S8  | 0   | R2  | R2  | S9  |
| 17 | R4    | 0   | 0   | R4  | R4  | R4  | R4  | R4  |
| 18 | R5    | 0   | 0   | R5  | R5  | R5  | R5  | R5  |
| 19 | R7    | 0   | 0   | R7  | R7  | R7  | R7  | R7  |
| 20 | 0     | S13 | 0   | 0   | 0   | 0   | 0   | S14 |
| 21 | 0     | S13 | 0   | 0   | 0   | 0   | 0   | S14 |
| 22 | 0     | S13 | 0   | 0   | 0   | 0   | 0   | S14 |
| 23 | 0     | S13 | 0   | 0   | 0   | 0   | 0   | S14 |
| 24 | 0     | 0   | S29 | 0   | S20 | S21 | 0   | 0   |
| 25 | 0     | 0   | R1  | S22 | R1  | R1  | S23 | 0   |
| 26 | 0     | 0   | R2  | S22 | R2  | R2  | S23 | 0   |
| 27 | 0     | 0   | R4  | R4  | R4  | R4  | R4  | R4  |

| 状态 | \$ | + | -  | *  | /  | (  | )  | n  |
|----|----|---|----|----|----|----|----|----|
| 28 | 0  | 0 | R5 | R5 | R5 | R5 | R5 | R5 |
| 29 | 0  | 0 | R7 | R7 | R7 | R7 | R7 | R7 |

GOTO表

| 状态 | E  | T  | F  |
|----|----|----|----|
| 0  | 1  | 2  | 3  |
| 1  | 0  | 0  | 0  |
| 2  | 0  | 0  | 0  |
| 3  | 0  | 0  | 0  |
| 4  | 10 | 12 | 11 |
| 5  | 0  | 0  | 0  |
| 6  | 0  | 3  | 15 |
| 7  | 0  | 3  | 16 |
| 8  | 0  | 17 | 0  |
| 9  | 0  | 18 | 0  |
| 10 | 0  | 0  | 0  |
| 11 | 0  | 0  | 0  |
| 12 | 0  | 0  | 0  |
| 13 | 24 | 12 | 11 |
| 14 | 0  | 0  | 0  |
| 15 | 0  | 0  | 0  |
| 16 | 0  | 0  | 0  |
| 17 | 0  | 0  | 0  |
| 18 | 0  | 0  | 0  |
| 19 | 0  | 0  | 0  |
| 20 | 0  | 12 | 25 |
| 21 | 0  | 12 | 26 |
| 22 | 0  | 27 | 0  |
| 23 | 0  | 28 | 0  |
| 24 | 0  | 0  | 0  |
| 25 | 0  | 0  | 0  |
| 26 | 0  | 0  | 0  |
| 27 | 0  | 0  | 0  |

| 状态 | E | T | F |
|----|---|---|---|
| 28 | 0 | 0 | 0 |
| 29 | 0 | 0 | 0 |

产生式编号说明

| 编号 | 产生式                   |
|----|-----------------------|
| 1  | $S \rightarrow E$     |
| 2  | $E \rightarrow E+T$   |
| 3  | $E \rightarrow T$     |
| 4  | $T \rightarrow T * F$ |
| 5  | $T \rightarrow T / F$ |
| 6  | $T \rightarrow F$     |
| 7  | $F \rightarrow (E)$   |
| 8  | $F \rightarrow n$     |

符号说明

- **Sx**: 移进并转移到状态x
- **Rx**: 按编号为x的产生式进行归约
- **ACC**: 接受输入
- **0**: 错误（空白项）

3 测试与分析

3.1 测试用例设计

设计了多种测试用例来验证程序的正确性：

- 合法表达式：`n+n*n`、`(n+n)*n`、`n/n-n` 等
- 非法表达式：`n+`、`+n`、`n++n`、`(n` 等

3.2 测试结果

3.2.1 LL(1)分析测试

输入表达式 `n+n*n` 的分析过程：

|   |       |         |       |
|---|-------|---------|-------|
| 1 | \$E   | n+n*n\$ | 1     |
| 2 | \$AT  | n+n*n\$ | 5     |
| 3 | \$ABF | n+n*n\$ | 10    |
| 4 | \$ABn | n+n*n\$ | match |
| 5 | \$ABF | +n*n\$  | 8     |

|    |       |        |        |
|----|-------|--------|--------|
| 6  | \$AT  | +n*n\$ | 2      |
| 7  | \$AT+ | +n*n\$ | match  |
| 8  | \$AT  | n*n\$  | 5      |
| 9  | \$ABF | n*n\$  | 10     |
| 10 | \$ABn | n*n\$  | match  |
| 11 | \$ABF | *n\$   | 6      |
| 12 | \$AB* | *n\$   | match  |
| 13 | \$AB  | n\$    | 5      |
| 14 | \$ABF | n\$    | 10     |
| 15 | \$ABn | n\$    | match  |
| 16 | \$ABF | \$     | 8      |
| 17 | \$AT  | \$     | 4      |
| 18 | \$A   | \$     | 4      |
| 19 | \$    | \$     | accept |

并且已经通过头歌平台所有样例测评：

测试结果

10/10 全部通过

▶ 测试集1

消耗内存92.69MB 代码执行时长：0.03秒

▶ 测试集2

消耗内存92.69MB 代码执行时长：0.02秒

▶ 测试集3

消耗内存92.69MB 代码执行时长：0.02秒

本关最大执行时间：120秒 本次评测耗时(编译、运行总时间)：1.518 秒

下一关

自测运行

评测

### 3.2.2 LR(1)分析测试

输入表达式 `n+n*n` 的分析过程：

|   |        |
|---|--------|
| 1 | shift  |
| 2 | 8      |
| 3 | shift  |
| 4 | 6      |
| 5 | shift  |
| 6 | 8      |
| 7 | shift  |
| 8 | 6      |
| 9 | accept |

并且已经通过头歌平台所有样例测评：

测试结果

10/10 全部通过

▶ 测试集1

消耗内存120.19MB 代码执行时长：0.04秒

▶ 测试集2

消耗内存120.19MB 代码执行时长：0.02秒

▶ 测试集3

消耗内存120.19MB 代码执行时长：0.02秒

本关最大执行时间：120秒 本次评测耗时(编译、运行总时间)：2.016 秒

上一关

自测运行

评测

### 3.3 错误处理能力

程序具备基本的错误处理能力：

- LL(1)分析器能够检测到语法错误并报告，并能通过添加同步信息进行错误恢复
- LR(1)分析器在遇到错误时能够停止分析

## 4 程序特点与改进建议

### 4.1 程序特点

1. **完整的LL(1)分析实现**：包括FIRST集、FOLLOW集计算和预测分析表构建
2. **基本的LR(1)分析框架**：实现了项目集规范族的构建
3. **模块化设计**：将不同功能模块分离，代码结构清晰
4. **错误处理机制**：具备基本的语法错误检测能力

### 4.2 改进方向

1. **增强错误恢复能力**：当前错误处理较为简单，可以增加更智能的错误恢复策略
2. **完善LR(1)分析器**：当前LR(1)分析器功能相对简单，可以进一步完善

## 5 总结

本次实验成功实现了LL(1)语法分析器和一个基本的LR(1)语法分析框架。通过实验，我深入理解了自顶向下和自底向上语法分析的基本原理，掌握了FIRST集、FOLLOW集的计算方法，以及预测分析表和LR(1)分析表的构建过程。

LL(1)分析器实现完整，能够正确分析算术表达式并给出详细的分析过程。LR(1)分析器虽然功能相对简单，但提供了良好的扩展基础。实验过程中遇到的挑战主要在于LR(1)项目集规范族的构建和分析表的生成，通过查阅资料和调试逐步解决了这些问题。

这次实验不仅加深了我对编译原理课程内容的理解，也提高了我的编程能力和问题解决能力。